

apply_call is generating an error message

<https://github.com/NVIDIA/cuda-quantum/issues/1130>

ROW 46

apply_call is generating an error message #1130

New issue

Closed



mmvandieren opened on Jan 23, 2024

...

Required prerequisites

- ☒ Make sure you've read the [documentation](#). Your issue may be addressed there.
- ☒ Search the [issue tracker](#) to verify that this hasn't already been reported. +1 or comment there if it has.
- ☒ If possible, make a PR with a failing test to give us a starting point to work on!

Describe the bug

I get the error message below about an invalid type being passed to the kernel, but it is a float which should be ok.

```
RuntimeError: Invalid argument type passed to kernel call (f64 != !quake.ref).
```

Steps to reproduce the bug

the code below generates the bug

```
qaoaProblem, qubit_0, qubit_1, alpha = cudaq.make_kernel(cudaq.qubit, cudaq.qubit, float)
qaoaProblem.cx(qubit_0, qubit_1)
qaoaProblem.rz(2*alpha, qubit_1)
qaoaProblem.cx(qubit_0, qubit_1)

from typing import List
# Approach 1: Define everything in one kernel

# Variable for the number of layers in the QAOA kernel
layer_count = 1 # This value can be increased, but at the cost of increased circuit depth

# Define a quantum kernel function for a parameterized quantum circuit
qaoaKernel, thetas = cudaq.make_kernel(List[float])

# Let's allocate a few qubits to our quantum circuit
# The number of qubits we'll need is the same as the number of vertices in our graph
qubit_count = 5

# Allocate n qubits to the kernel.
qreg = qaoaKernel.qalloc(qubit_count)

# Apply a Hadamard gate to each qubit
qaoaKernel.h(qreg)

# The QAOA circuit for Max Cut depends on the structure of the graph
# Each node is associated with a qubit. We'll sort the nodes and
# use the index of a vertex in this sorted list to determine the qubit
# associated with that vertex.
nodes = [2,5,7]
edges = [(2,5), (5,7)]

# Loop over the layers
for i in range(layer_count):
    # Problem unitary
    for edge in edges:
        qubit_u = qreg[nodes.index(edge[0])]
        qubit_v = qreg[nodes.index(edge[1])]
        # I'm getting an error message with the next line
        qaoaKernel.apply_call(qaoaProblem, qubit_u, qubit_v, thetas[i])
```

Expected behavior

the kernel called `qaoaProblem` should be applied to the 2 given qubits with the parameter value. The parameter value seems to generate an error message.

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



```

qaoaKernel.h(qreg)

# The QAOA circuit for Max Cut depends on the structure of the graph
# Each node is associated with a qubit. We'll sort the nodes and
# use the index of a vertex in this sorted list to determine the qubit
# associated with that vertex.
nodes = [2,5,7]
edges = [(2,5), (5,7)]

# Loop over the layers
for i in range(layer_count):
    # Problem unitary
    for edge in edges:
        qbitu = qreg[nodes.index(edge[0])]
        qbitv = qreg[nodes.index(edge[1])]
        # I'm getting an error message with the next line
        qaoaKernel.apply_call(qaoaProblem, qbitu, qbitv, thetas[i])

```

Expected behavior

the kernel called `qaoaProblem` should be applied to the 2 given qubits with the parameter value. The parameter value seems to generate an error message.

Is this a regression? If it is, put the last known working version (or commit) here.

Not a regression

Environment

- CUDA Quantum version: container 40733b768160
- Python version: 3.10.12
- C++ compiler: unknown
- Operating system: linux

Suggestions

```
Type argType = otherFuncCloned.getArgumentTypes()[i];
```

should be

```
Type argType = otherFuncCloned.getArgumentTypes()[i++];
```

the `i` was not being incremented

https://github.com/NVIDIA/cuda-quantum/blob/main/runtime/cudaq/builder/kernel_builder.cpp#L295



amccaskey mentioned this on Jan 23, 2024

[Fix issue with argument checking on kernel_builder apply_call #1131](#)



bmhowe23 on Jan 24, 2024

Collaborator ...

Fixed via [#1131](#)



bmhowe23 closed this as [completed](#) on Jan 24, 2024

Fix issue with argument checking on kernel_builder apply_call #1131

<> Code

Merged amccaskey merged 1 commit into NVIDIA:main from amccaskey:fix1130 on Jan 24, 2024

Conversation 4 Commits 1 Checks 148 Files changed 2 +45 -1

 amccaskey commented on Jan 23, 2024

Collaborator

#1130

 Fix issue with argument checking on kernel_builder apply_call

✓ f88ce89

 copy-pr-bot commented on Jan 23, 2024

bot

This pull request requires additional validation before any workflows can run on NVIDIA's runners.

Pull request vetters can view their responsibilities [here](#).

Contributors can view more details about this message [here](#).

amccaskey requested review from anthony-santana, 1tnguyen and bmhowe23 last year

 amccaskey commented on Jan 23, 2024 • edited by github-actions

Collaborator Author

/ok to test

Command Bot: Processing...

1 1

 github-actions commented on Jan 23, 2024

bot

CUDA Quantum Docs Bot: A preview of the documentation can be found [here](#).

 github-actions pushed a commit that referenced this pull request on Jan 23, 2024

Docs preview for PR #1131.

59be3f9

amccaskey enabled auto-merge (squash) last year

 bmhowe23 approved these changes on Jan 24, 2024

View reviewed changes

Reviewers

 bmhowe23 ✓

 anthony-santana

 1tnguyen

Assignees

No one assigned

Labels

bug fix

Projects

None yet

Milestone

release 0.7.0

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

3 participants

  